Permutations and Operator Combinations

Generating arrangements in place

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 15 Thursday, October 22, 2026

Today

1. **Counting arrangements**: the dinner-party problem and $n!$.
2. **Generating** every arrangement with recursion: choose, explore, undo.
3. What goes wrong when you **forget the undo**.
4. **Operator combinations**: the same pattern, a different loop.
5. How the two pieces fit together in Make 24 (Project 3).

- **Exercise 7 (Permutation Trace)** is posted today and due **Tuesday, Nov 3**. Today's ideas are the toolbox for it; the handout has the exact tasks.
- Midterm: Tue, Oct 27, in class. Today's material is **not** on it.
- **Announcement:** the **midterm cheat sheet was corrected** on Oct 20. Please **re-download** it from the course schedule.

Question: The Dinner Party

Three guests, **A**, **B**, **C**, sit in three chairs in a row.

(a) List every seating arrangement. How many are there?
(b) Explain the count without listing: how many guests can take chair 1? Then chair 2? Then chair 3?
(c) How many arrangements for 4 guests? For 10 guests?

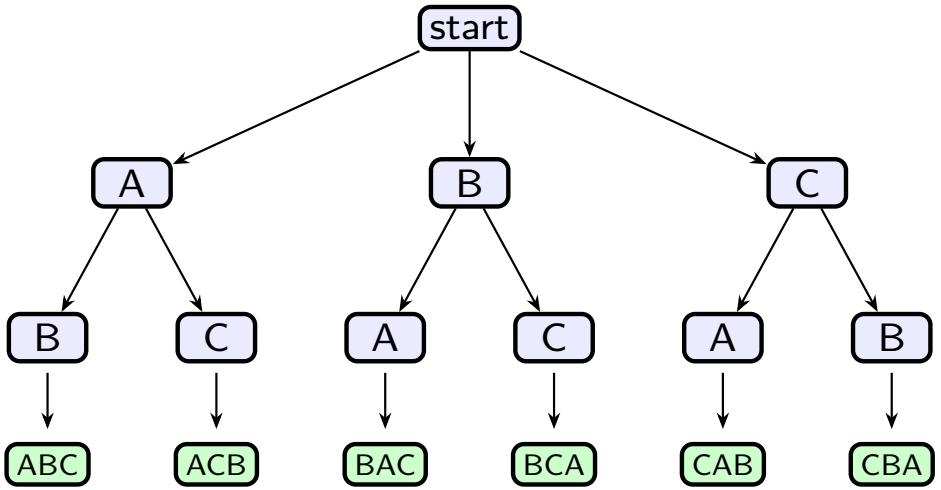
Question: The Dinner Party

Three guests, **A**, **B**, **C**, sit in three chairs in a row.

(a) List every seating arrangement. How many are there?
(b) Explain the count without listing: how many guests can take chair 1? Then chair 2? Then chair 3?
(c) How many arrangements for 4 guests? For 10 guests?

(a) ABC, ACB, BAC, BCA, CAB, CBA: **6**.
(b) Chair 1: 3 choices. Chair 2: 2 guests left. Chair 3: 1 guest left. $3 \times 2 \times 1 = 6$.
(c) $4! = 24$; $10! = 3\,628\,800$. In general $n! = n \times (n-1) \times \cdots \times 1$.

The Arrangement Tree



Read the tree **top to bottom**: each level is one chair.

Level 1: 3 branches. Level 2: 2 branches under each. Level 3: 1 branch.

$3 \times 2 \times 1 = 6$ leaves. This tree is the **state space**. A recursive function walks it **depth first**, visiting leaves left to right.

Generating Them: Swap, Recurse, Undo

Trick: keep the guests in one array. Chair `k` onward are *still unseated*. To fill chair `k`, swap each unseated guest into it.

```
void printAll(std::string& s, int k) {
    if (k == static_cast<int>(s.size())) {  // every chair filled
        std::cout << s << "\n";
        return;
    }
    for (int i = k; i < static_cast<int>(s.size()); ++i) {
        std::swap(s[k], s[i]);              // 1. choose
        printAll(s, k + 1);                 // 2. explore
        std::swap(s[k], s[i]);              // 3. undo
    }
}
// std::string s = "ABC"; printAll(s, 0);
```

Question: In what order are the six strings printed? (Trace by hand before the next slide.)

Tracing `printAll(s, 0)` with `s = "ABC"`

#	call (`k`), loop `i`	after swap	prints	after undo
1	k=0, i=0; then k=1, i=1	ABC		
2	k=1, i=2	ACB		
3	k=0, i=1; then k=1, i=1	BAC		
4	k=1, i=2	BCA		
5	k=0, i=2; then k=1, i=1	CBA		
6	k=1, i=2	CAB		

Tracing `printAll(s, 0)` with `s = "ABC"`

#	call (k), loop i	after swap	prints	after undo
1	k=0, i=0; then k=1, i=1	ABC	ABC	ABC
2	k=1, i=2	ACB	ACB	ABC
3	k=0, i=1; then k=1, i=1	BAC	BAC	BAC
4	k=1, i=2	BCA	BCA	BAC
5	k=0, i=2; then k=1, i=1	CBA	CBA	CBA
6	k=1, i=2	CAB	CAB	CBA

Each time `k=0` finishes an iteration, the undo restores `ABC` before the next `i`. Output order: **ABC, ACB, BAC, BCA, CBA, CAB**. It differs from the tree order only in the last two leaves (the tree listed CAB before CBA).

Question: What If We Forget the Undo?

Remove the **second** `std::swap` (the undo). Same input `"ABC"`. What is printed?

(A) The same six strings, in the same order.

(B) Six strings, but some repeat and some arrangements are missing.

(C) Nothing is printed.

(D) The program crashes immediately.

Question: What If We Forget the Undo?

Remove the **second** `std::swap` (the undo). Same input `"ABC"`. What is printed?

(A) The same six strings, in the same order.

(B) Six strings, but some repeat and some arrangements are missing.

(C) Nothing is printed.

(D) The program crashes immediately.

Answer: (B). It prints **ABC, ACB, CAB, CBA, ABC, ACB**. Without the undo, the array is left in the *wrong* order after each recursive call, so the next `swap` starts from a scrambled state. Missing: BAC and BCA. Repeated: ABC and ACB.

Moral: choose and undo must **mirror** each other exactly.

Copying vs. In-Place

	Copy the state	**Change one state, then undo**
How	pass a new vector/string **by value** into each call	pass **by reference**; swap, recurse, swap back
Undo?	automatic: the copy is discarded	**you** must write it
Cost	copying at every call	two $O(1)$ swaps per branch; no string copy
Danger	slow for large states	forgetting or mis-ordering the undo

Analogy. Copying prints a fresh seating chart for every experiment; in-place erases and rewrites one chart: faster, but erase carefully.

By value protects the *caller* from a child's changes. It does **not** undo changes the caller makes to its own copy inside its loop.

Rule of thumb: copy while you learn; go in-place when the state is large or speed matters.

Duplicates: What About `"AAB"`?

Question. How many *distinct* arrangements does `"AAB"` have? How many strings does `printAll` print?

Duplicates: What About `"AAB"`?

Question. How many *distinct* arrangements does `"AAB"` have? How many strings does `printAll` print?

Distinct: **3** (AAB, ABA, BAA). `printAll` prints **6** lines:

AAB, ABA, AAB, ABA, BAA, BAA

The function treats the two A's as *different people*. Each distinct arrangement appears $2! = 2$ times.

Choices: accept the repeats (simple), filter them afterward (e.g. keep results in a `std::set`), or prune repeated branches while searching. Read your Exercise 7 handout to see what it asks for.

Operator Combinations: A Different Loop

Problem. Numbers 1 2 3 in order. Fill the **two gaps** with $+, -, *, /$. Print every expression.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n";
        return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();          // remember the length
        expr += op;                             // 1. choose
        expr += std::to_string(v[k]);
        ops(v, k + 1, expr);                    // 2. explore
        expr.resize(len);                       // 3. undo
    }
}
// std::string e = "1"; ops({1,2,3}, 1, e);
```

Note: it prints expression *strings* only: no evaluation, parentheses, or division-by-zero check.

Question: How many lines? What are the first four?

Operator Combinations: A Different Loop

Problem. Numbers 1 2 3 in order. Fill the **two gaps** with $+, -, *, /$. Print every expression.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n";
        return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();          // remember the length
        expr += op;                             // 1. choose
        expr += std::to_string(v[k]);
        ops(v, k + 1, expr);                    // 2. explore
        expr.resize(len);                       // 3. undo
    }
}
// std::string e = "1"; ops({1,2,3}, 1, e);
```

Note: it prints expression *strings* only: no evaluation, parentheses, or division-by-zero check.

Question: How many lines? What are the first four?

16 lines. First four: 1+2+3, 1+2−3, 1+2*3, 1+2/3; the **last** gap changes fastest.

Putting the Counts Together

A Make 24 solver that tries **every** order ($n!$), every operator choice (4^{n-1}), and every parenthesization (Catalan numbers 2, 5, 42):

n	$n!$	4^{n-1}	parenth.	**total**
3	6	16	2	
4	24	64	5	
6	720	1 024	42	

Question: Fill the last column: multiply the three columns.

Putting the Counts Together

A Make 24 solver that tries **every** order ($n!$), every operator choice (4^{n-1}), and every parenthesization (Catalan numbers 2, 5, 42):

n	$n!$	4^{n-1}	parenth.	**total**
3	6	16	2	192
4	24	64	5	7 680
6	720	1 024	42	30 965 760

Question: Fill the last column: multiply the three columns.

A *different enumeration* from last class's 3888, counted before excluding division by zero. It keeps both operand orders for $+$ and $\times$; last class's pair search used one orientation (and can reach one expression by different orders of independent combinations). Neither counts distinct results; with repeated values, $n!$ counts input *positions*. Either way, six numbers means tens of millions of checks: we will be clever on Oct 29.

Common Bugs

- **Undo that does not match the choice** (wrong indexes, wrong order, or missing).
- Passing the state **by value** when you meant by reference (changes vanish), or by reference when you needed a copy (changes leak between branches).
- Looping from 0 instead of from `k`: already-seated guests get swapped again.
- **Printing or recording the result at the wrong depth**: do it only in the base case.
- Forgetting that results from the loop come out in **depth-first order**, not sorted order.
- Storing a *reference* to a changing state instead of a **copy** of the finished result.

Practice

1. How many arrangements of 5 different letters?
2. In `printAll("ABCD", 0)`, how many lines are printed? How many times is the base case reached?
3. In `printAll(s, 0)` with `s = "ABC"`, the call with `k = 1` runs its loop for `i` $= 1$ and 2. How many values does `i` take when `k = 2`?
4. How many expressions does `ops` print for 5 numbers?

Practice

1. How many arrangements of 5 different letters?
2. In `printAll("ABCD", 0)`, how many lines are printed? How many times is the base case reached?
3. In `printAll(s, 0)` with `s = "ABC"`, the call with `k = 1` runs its loop for `i` = 1 and 2. How many values does `i` take when `k = 2`?
4. How many expressions does `ops` print for 5 numbers?

1. $5! = 120$.
2. $4! = 24$ lines; the base case is reached 24 times, once per leaf.
3. One: $i = 2$ only. A single chair left has a single guest.
4. $4^4 = 256$.

Summary and Preview

- *n distinct* items have $n!$ arrangements; the search tree has one level per chair.
- Recursive enumeration: **choose, explore, undo**, and the undo mirrors the choose.
- Operator combinations use the same pattern, with 4 choices per gap.
- The counts multiply: orders $\times$ operators $\times$ parentheses.

Coming up

Oct 27: Midterm (through DFS/BFS).
Oct 29: pruning and early termination: how to avoid walking the whole tree.
Nov 3: Exercise 7 due.